

AsymptoticAnalysis

An incremental audit of the native boundary

Wolfram Language capabilities, newly reproduced interface defects, held-request semantics, and a focused repair plan

Repository	https://github.com/VladimirReshetnikov/Asymptotic	Prin-
Audited revision	6687962f3c858a4f93623cfc496f33e35c6763d4	
Package	AsymptoticAnalysis, version 1.8.0	
Review date	September 9, 2026, America/Los_Angeles	
Independent kernel	Wolfram Language 15.0.0, Linux x86 (64-bit), May 6, 2026	
Prior-work baseline	Eighteen external reviews; the current consolidated implementation register	

cipal result. Three native-interface defects are reproduced on the pinned package: configured defaults can be ignored; standard string option spellings can be rejected; and an expansion variable that shares an option name can lose its variable and specification metadata. These are distinct from the previously recorded mathematical remainder, composition, export, and certificate findings.

Evidence boundary. Ninety tests in four existing native-interface test files passed in this session. Additional focused probes demonstrate the defects. This is not a full-suite acceptance, a proof of all mathematical engines, or a certification of universal native compatibility. The accompanying repair candidate also corrected five focused probes; its wider acceptance is separately delimited.

Contents

1	Executive assessment	3
1.1	New findings	3
1.2	Recommendation	3
2	Snapshot, evidence, and non-duplication	4
2.1	Immutable source and runtime identity	4
2.2	What was actually examined	4
2.3	Prior-review filtering	5
3	The mathematical and software architecture	6
3.1	The analytic object is a contract, not just a polynomial	6
3.2	Specialized scales are complementary, not one unrestricted ring	7
3.3	Three evidence channels must remain separate	8
3.4	The new native result contract	8
4	Comparison with Wolfram Language built-ins	8
4.1	Use the complete native baseline	8
4.2	Order conventions must be aligned before comparing coefficients	10
4.3	Live option lists are more reliable than guessed compatibility	10
4.4	Comparisons should test transformations, not only example functions	11
5	N1: configured defaults are ignored at the routing boundary	11
5.1	Minimal reproductions	11
5.2	Source-level cause	12
5.3	Why the mathematical answer can be valid and the API still wrong	12
5.4	A safe default-resolution policy	13
5.5	Acceptance criteria	13
6	N2: equivalent option spellings receive incompatible treatment	13
6.1	The observed native differential	13
6.2	Exact held syntax is being used for semantic identity	14
6.3	Canonicalize identity without evaluating values	14
6.4	Tests beyond the smallest example	14
7	N3: specification parsing confuses variable names with option names	15
7.1	An expansion variable named Method is legal	15
7.2	The classifier ignores positional grammar	15
7.3	A positional prefix parser	16
7.4	Why role information should be retained	16

8	A request-semantics framework for the three repairs	16
8.1	The missing effective-request layer	16
8.2	Three bounded invariants	17
8.3	Do not overpromise effect equivalence	17
8.4	A practical acceptance matrix	18
9	Performance, representation, and development opportunities	18
9.1	P-N1: native normalization is eagerly materialized	18
9.2	Budget only the work actually owned by the wrapper	19
9.3	Native operations without analytic promotion	20
9.4	The path toward complete native coverage	20
10	Patch candidate and artifact design	20
10.1	What the candidate changes	20
10.2	What it deliberately does not change	21
10.3	Validation status of the candidate	21
10.4	How to reproduce and integrate	22
11	Validation results, limits, and development sequence	22
11.1	Existing native-interface tests executed here	22
11.2	Useful negative results	22
11.3	A focused sequence with objective exit criteria	23
12	Conclusion	23
A	Archive contents and evidence interpretation	24
B	Reference notes	24

1 Executive assessment

The repository has developed into two cooperating products. One is a specialized real asymptotic calculus with explicit remainder descriptors, branch information, coefficient blocks, and retained reconstruction data. The other is a compatibility wrapper around the built-in **Series** and **Asymptotic** engines. The former is substantially more than an alternative printing format for a Taylor series. The latter is substantially less than a completed proof that every supported native request is transparently accepted. The current repository documentation makes this distinction explicitly [1, 5].

The strongest incremental opportunity is no longer to repeat the first review wave’s warnings about dense export, hidden assumptions, or pure-remainder fractional powers. Those issues have specific status entries, and several now have scoped repair evidence. The compatibility boundary deserves independent tests of its own. It sits before coefficient computation, so it can mishandle an otherwise trivial example even when the underlying mathematics is completely sound.

1.1 New findings

ID	Finding	Evidence	User-visible consequence
N1	Configured defaults are not consistently part of dispatch	Executed	<code>SetOptions[... , "Backend" -> "Series"]</code> is ignored; a configured four-block goal can become a one-term native request. The alias has a separate ineffective default list.
N2	Equivalent native option spellings are classified differently	Executed	<code>"Assumptions" -> a > 0</code> produces <code>NativeOptionConflict</code> , although the corresponding native call uses the assumption correctly.
N3	Option-name membership is mistaken for argument role	Executed	A native expansion in the legitimate variable <code>Method</code> computes its native answer, but loses its recognized specification and variable; numerical application of the wrapper fails.

These findings should be treated as interface correctness defects, not as three newly discovered false asymptotic theorems. In particular, the one-term result in N1 is still a valid leading approximation to the example. It is the wrong response to the configured request, and it changes result kind and order semantics. That distinction matters for severity and for a repair that does not unnecessarily modify coefficient or remainder algorithms.

A separate performance opportunity, P-N1, concerns the new wrapper’s eager conversion of every native result to **Normal** even when only `"NativeResult"` is wanted. A successful size experiment records native, normalized, and wrapper `ByteCount` values at orders 100 and 1000, with matching outputs. This establishes expression-size overhead, not a measured slowdown or a peak-memory result.

1.2 Recommendation

Repair request interpretation before enlarging the native input matrix. Define an effective request that preserves the source’s held evaluation, distinguishes specifications from options by grammar, normalizes option identity without evaluating option values, and includes the configured defaults

relevant to routing. Then run the native engine and wrap its result without claiming additional analytic evidence.

The supplied patch candidate implements a narrow version of those changes. It leaves the analytic jet engines, special-function formulas, certificate arithmetic, and native-result remainder policy alone. It should be evaluated as an integration candidate with explicit acceptance criteria, not imported as an unquestioned fix. Section 10 explains the policy choices and limitations.

2 Snapshot, evidence, and non-duplication

2.1 Immutable source and runtime identity

The review uses commit `6687962f3c858a4f93623cfc496f33e35c6763d4`, whose commit time is September 10, 2026, 00:18:17 UTC, or September 9 in the user’s Pacific timezone. The commit records successful loading of the published standalone URL on the maintainer’s Windows kernel. That upstream record is separate from the Linux observations reported here [2, 1].

The root `AsymptoticAnalysis.wl` is the generated, self-contained distribution. The canonical source lives under `src/Kernel/`. The current public context is `AsymptoticAnalysis`; the ordinary expansion object is `GeneralizedSeries`. Historical review packages retain older names and paths. Replacing those names indiscriminately in old patches is not a substitute for checking the current implementation [4].

Independent package execution used the following kernel identification:

```
"15.0.0 for Linux x86 (64-bit) (May 6, 2026)"
"Linux-x86-64"
```

The maintained package declares Wolfram Language 15.0 or later; the maintainer’s focused acceptance records generally identify 15.0.1 for Windows. Agreement on the selected Linux tests is useful additional evidence, but does not make the two environments interchangeable for all symbolic evaluation, messages, or performance.

The connector’s kernels are stateless across evaluations. Each successful package probe downloaded the pinned standalone source as text, exported it to a temporary local file, and loaded that file with `Get`. The supplied loader supports an already downloaded local standalone file for offline replay. Intermittent connector failures occurred. A failed connector invocation is not counted as a package failure or as a completed experiment.

2.2 What was actually examined

The detailed source inspection concentrates on the new native dispatcher and result constructor; the core public contracts, assumption boundary, result access and loader organization; the exact interval-certificate implementation; and the focused native-interface tests. The README, native compatibility plan, implementation register, and relevant prior-review articles supply the wider architecture and historical context.

This is a broad capability assessment with a deep incremental audit at the changed interface boundary. It does not claim that every line of every specialized coefficient engine has been independently re-proved. The source-coverage file in the archive distinguishes detailed inspection, contract-level inspection, execution, and architecture-only context. That is more informative than an unqualified claim to have “verified the repository.”

The evidence has four levels:

1. **Observed on the pinned package:** the focused N1–N3 outputs and the ninety-test native-interface run.
2. **Established from inspected source:** the dispatcher branches, exact option-key comparisons, variable extraction, and eager native normalization.
3. **Derived reasoning:** the interpretation invariants and why the counterexamples violate them.
4. **Proposed work:** the patch, the expanded acceptance matrix, lazy native views, and the remaining native-coverage policy.

Only the first category is described as a current native reproduction. A proposed regression is not represented as a passing test unless its execution is recorded.

2.3 Prior-review filtering

The exclusion baseline is not a single old article. The repository contains eighteen reports in two waves, consolidated into 123 identified finding entries and shared work items in the `implementation` register. Reports 1–6, 8, and 9 concern `07a9781...`; report 7 concerns `75de875...`; reports 10–18 concern `921387e...`. Those snapshots are older than the audited revision; the register separately records native compatibility work and subsequent repairs [4, 3].

The register was used to remove already recorded defects from the new-finding list. A targeted search was also run over eighteen primary article files or finding chapters, including the standalone first report and selected chapters for reports 7 and 15. The search covered configured defaults, backend routing, option-name and option-key classification, string option spellings, and the new specification helper. No `SetOptions`, `nativeSpecification`, or “string option” match occurred in those selected texts. This supports the filter; keyword absence alone is not a proof of novelty. The dated source boundary and the current implementation register provide the more important context.

A representative exclusion crosswalk is useful without reproducing the earlier reviews:

Existing work	Why it is not a new finding here	Relationship to this review
C01, C03, C06	Dense native views and logarithmic import/export tails already have detailed findings and scoped repairs.	Native delegation is a different result path. N1–N3 do not allege loss of logarithmic remainder information.
C02, C04, C07	Pure-remainder branches, nonlinear boundary degrees, and real-coefficient admission already have repair records.	The patch intentionally leaves these analytic rules unchanged.
C05, C19	Assumption capture and certificate-accuracy planning already have substantive repairs and evidence.	A configured <i>routing</i> default being ignored is not the old ambient-assumption certificate flaw.
C08, C09	Recipe precision and stored-versus-explicit inverse coefficient power already have open entries.	N1 concerns the expansion dispatcher’s public defaults, not the older coefficient overload.
C14–C18	Equal exponents, diagonal composition, formal source admission, native index limits, and target geometry are already recorded.	These remain outside the new-defect count. No incidental refusal is relabeled as a proof that these obligations are solved.

Existing work	Why it is not a new finding here	Relationship to this review
P01–P08; D01–D02	Sparse-work budgets, provenance size, profiling, and common request/result contracts are existing proposals.	P-N1 identifies a particular newly introduced native materialization site; the request proposal is narrowed to three reproduced interpretation failures.
B01–B04	Complete native coverage, a second-backend search, and broader matrix evidence are explicitly unfinished.	Their mere absence is not announced as a new discovery. The concrete standard-option counterexample supplies a new failing matrix row.

The resulting article is deliberately not a second catalogue of the register. Earlier findings matter as constraints on a safe repair and as reasons not to overstate current completion.

3 The mathematical and software architecture

3.1 The analytic object is a contract, not just a polynomial

The ordinary package representation works in a positive local coordinate $u \rightarrow 0^+$. At a finite right-hand endpoint, $u = x - x_0$; on a left-hand approach, $u = x_0 - x$; at a signed infinite endpoint a reciprocal coordinate is used. A typical retained expansion has the form

$$f(x) = \sum_{\alpha < h} u^\alpha P_\alpha(\log u) + O\left(u^\beta (1 + |\log u|)^k\right), \quad (1)$$

with finitely many retained real exponents and polynomial logarithmic coefficients. The public `PowerLogRemainder[u,beta,k]` is an inert descriptor of the displayed error class, not a floating-point estimate [7, 1].

The exponent cutoff is exclusive. A complete polynomial at one exponent is one block even when it contains several logarithmic monomials. A term goal is consequently not a request for the same number of ordinary summands. These are important design choices for resonant correction weights: terms at the same asymptotic weight belong together, and discarding half of a block can misrepresent both cancellation and the next error scale.

The ordinary inverse model uses a translated and normalized structure such as

$$y = y_0 + au^p \left(1 + \sum_{j=1}^m u^{\delta_j} B_j(\log u) \right), \quad \delta_j > 0. \quad (2)$$

The branch and coordinate information determine the allowed inverse uniformizer. The package provides direct Lagrange-style coefficient construction, grouped and Newton routes, exact-termination handling, coefficient queries, retained state, and residual checks. These capabilities concern the supported model and its evidence; they are not a universal solver for arbitrary inverse functions [7, 1, 4].

A finite expression extracted by `Normal` loses the package remainder and much of the reusable context. The object is therefore not merely cosmetic. Addition, multiplication, composition, powers, observables, truncation, refinement, and differentiation have obligations about the associated uncertainty. In particular, a magnitude Big-O estimate alone is insufficient to justify differentiating that estimate. The public differentiation API explicitly distinguishes derivative evidence from a plain error magnitude [7].

3.2 Specialized scales are complementary, not one unrestricted ring

The loader composes a substantial collection of specialized modules. The following grouping describes the architecture rather than claiming exhaustive domain coverage for every head.

Layer	Representative modules	Distinctive responsibility
Ordinary exact jets	<code>AsymptoticAnalysis.wl</code> , <code>ExactTermination.wl</code> , <code>IncrementalInverse.wl</code>	Ordered real powers, logarithmic coefficient blocks, finite models, inverse coefficients and retained work.
Coordinates and core inversion	<code>CoordinateInverse.wl</code> , <code>SourceCoordinates.wl</code> , <code>LambertInverse.wl</code> , <code>CorePerturbation.wl</code>	Source/target charts and exact cores; perturbations must be interpreted in the correct coordinate.
Gamma and Barnes families	Forward, inverse, inverse-check, and operation modules for Gamma/Barnes	Exact carriers or Lambert cores with structured correction blocks rather than expansion of every factor into one ordinary polynomial.
Logarithmic, exponential, and flat scales	<code>LogarithmicScales.wl</code> , <code>ExponentialForward.wl</code> , <code>FlatSectors.wl</code> , <code>FlatSectorOperations.wl</code>	Hierarchies and separate exponentially small sectors; cutoff meanings are scale-specific.
Oscillatory and special functions	<code>FourierCoefficients.wl</code> , special-function adapters, parameterized and Dirichlet modules, native special ingress	Endpoint-specific identities, carriers, domain conditions, and admissible tail descriptions.
Operations and replay	<code>SeriesOperations.wl</code> , <code>SeriesArithmetic.wl</code> , <code>SeriesEnvelopeArithmetic.wl</code> , refinement modules	Precision propagation, compatible envelopes, expression normalization, and retained-source reconstruction.
Independent evidence	<code>InverseCertificates.wl</code> , numerical and residual checks	Separate finite-model residuals, numerical comparisons, and exact interval proofs.
Native compatibility	<code>NativeCompatibility.wl</code>	Held request dispatch and a distinct loss-preserving native result kind.

For example, the positive increasing Gamma inverse retains an exact Lambert core and orders complete reciprocal-logarithmic polynomials by inverse powers of that core. Barnes inversion uses a related but different core and logarithmic coefficient variable. Forward Gamma/Barnes products and elementary exponentials can retain a large exact prefactor and expand only a correction bracket. In those cases a cutoff in the bracket is not an absolute power cutoff of the entire displayed expression [1, 7].

The supported Bessel, Airy, error-integral, incomplete Gamma, zeta/polylogarithm, hypergeometric, and elliptic cases are endpoint- and parameter-dependent. The README explicitly does not promise every special function, every branch, or every simultaneous parameter limit. A comparison that describes such restrictions as newly discovered unimplemented functions would misread the

stated scope.

3.3 Three evidence channels must remain separate

The current APIs expose three different kinds of checking. `InverseResidual` composes against an appropriate stored finite model and reports a normalized residual. `InverseNumericalCheck` compares an approximation with a numerical reference. `InverseCertificate` can prove a unique root enclosure using exact rational interval endpoints, directed dyadic rounding, and explicit elementary-function tail bounds [7, 8].

The certificate implementation inspected here requires the whole closed verification interval to satisfy retained source-domain conditions. It separates the derivative from zero, establishes existence through residual containment or endpoint signs, and intersects a sharper mean-value enclosure. Its reported scope is a root of the stored explicit function, not an enclosure of all unspecified functions represented by an input remainder. This is a valuable and carefully delimited facility.

The review does not reinterpret a small residual, a numerically vanishing error, or a native `SeriesData` tail as an interval certificate. Nor does the availability of the new native result kind justify bypassing the real-domain checks in these analytic evidence paths. The existing native contract tests explicitly reject such operations when their required analytic input evidence is absent [9].

3.4 The new native result contract

A native result stores the complete engine output, its held delegated request, original arguments, recognized specifications, ambient assumptions, runtime provenance, and a normalization view. Its important fields include

```
"Kind" -> "Native"
"Scale" -> "Native"
"NativeBackend" -> "Series" (* or "Asymptotic" *)
"NativeResult" -> result
"Remainder" -> Missing["NativeContract"]
"Exact" -> Missing["NotEstablished"]
```

The missing values mean that the wrapper has not supplied the package's analytic theorem. They do *not* mean that the built-in result has no mathematical semantics. `Series` supplies its native formal order; `Asymptotic` has a documented asymptotic contract. Preserving those contracts separately is the correct direction [5, 11, 14].

The current native wrapper is intentionally not closed under the package's analytic arithmetic. Native values can be extracted for native operations, but adding them to an analytic package object must not silently upgrade the native tail. The focused native contract suite covers both operand orders, several unary operations, composition, normalization, and inverse evidence refusals. This distinction was exercised in the successful ninety-test run.

4 Comparison with Wolfram Language built-ins

4.1 Use the complete native baseline

It would be inaccurate to compare this repository only with an elementary Taylor expansion around zero. The official `Series` interface includes fractional and Laurent expansions, successive variable specifications, and symbolic coefficients; `InverseSeries` reverts suitable `SeriesData`

objects. **Asymptotic** handles a considerably wider family of elementary and special-function scales and can work with expressions involving integrals, transforms, or differential-equation solutions. **AsymptoticSolve** handles implicit equations, systems, real-domain restrictions, and several kinds of asymptotic branches [11, 13, 14, 15].

Consequently, irrational exponents, logarithms, or an implicit equation are not by themselves proof of a native limitation. The meaningful questions are: was the specific request computed on the named kernel; did the same branch and order convention apply; what result representation was returned; and which evidence survives subsequent operations?

Task	Built-in baseline	Package contribution and boundary
Local expansion	Series , with its native order and coefficient representation.	Real-power/logarithmic block calculus, explicit analytic remainder metadata, retained coordinate and source data. Native mode delegates instead of requiring this representation.
Ordinary local inversion	InverseSeries ; AsymptoticSolve for equations and branches.	Selected real branch models, direct coefficient access, retained inversion state, and specialized nonordinary cores. A native inversion failure must be demonstrated, not inferred from syntax.
Large-argument or exotic scales	Asymptotic and the wider asymptotic family.	Explicit exact carriers, selected Lambert/Gamma/Barnes structures, logarithmic and flat-sector operations with package-specific contracts. Neither side is accurately summarized as “only powers.”
Complex or symbolic-native input	Native engines accept many inputs outside the package’s real analytic coefficient model.	Explicit native modes preserve the complete result without pretending that it is a real analytic package jet. Automatic coverage remains partial.
Multivariable work	Successive native series and native implicit-system asymptotics.	Delegation can preserve these outputs. They are not automatically uniform package error bounds along arbitrary parameter paths.
Algebra and calculus	Native SeriesData arithmetic, ComposeSeries , InverseSeries , and Normal .	Arithmetic transports package remainder and domain evidence where supported. Differentiation has a separate derivative-evidence requirement.
Numerical checking	Numerical equation solvers and interval facilities exist as general tools.	A dedicated inverse-check workflow and a scoped exact-rational certificate implementation for supported functions and intervals. No universal certificate claim follows.
Reproducibility	A native call is sensitive to its kernel, options, assumptions, and surviving symbol definitions.	Results retain request/provenance data, but not an immutable snapshot of every external definition. N1–N3 affect this wrapper-level fidelity.

4.2 Order conventions must be aligned before comparing coefficients

The following two commands intentionally ask different questions:

```
AsymptoticExpansion[Exp[x], {x, 0, 3}, "Backend" -> "Package"]
AsymptoticExpansion[Exp[x], {x, 0, 3}, "Backend" -> "Series"]
```

The first retains powers strictly below three, while the second delegates native order three. In the executed baseline their finite expressions are respectively

$$1 + x + \frac{x^2}{2}, \quad 1 + x + \frac{x^2}{2} + \frac{x^3}{6}.$$

This difference is documented and is not itself a bug. It becomes a bug when an explicit configuration selecting one interpretation is ignored, as in N1.

For ordinary integer-power Taylor examples, a package exclusive cutoff h and native **Series** order $h - 1$ are a useful matched pair. That shortcut does not universally translate real exponents, logarithmic blocks, exact carriers, or scale-relative goals. A benchmark must record the achieved representation and remainder, not merely compare the same integer supplied to two differently defined interfaces.

A rule-form request with no term goal has an intentional native leading-order route. For example, the current wrapper returns the native leading term for **Sin[x]**, $x \rightarrow 0$. Again, this documented default is not criticized here. A configured nondefault term goal needs to participate in deciding whether that default rule applies.

4.3 Live option lists are more reliable than guessed compatibility

The independently executed kernel returned:

```
Options[Series]
(* {Analytic -> True, Assumptions :> $Assumptions,
    SeriesTermGoal -> Automatic} *)

Options[Asymptotic]
(* {AccuracyGoal -> Automatic, Assumptions :> $Assumptions,
    GenerateConditions -> Automatic, GeneratedParameters -> None,
    Method -> Automatic, PerformanceGoal :> $PerformanceGoal,
    PrecisionGoal -> Automatic, SeriesTermGoal -> 1,
    WorkingPrecision -> Automatic} *)
```

These lists explain why copying a union of options does not define a single common default semantics. **SeriesTermGoal** is a shared key with different defaults and different contexts of use. The native engines also have different supported option sets. The compatibility layer must choose a backend and a policy; it cannot manufacture a correct policy merely by concatenating the option lists.

The documentation for **OptionValue** adds two less visible obligations. Option names are compared without their contexts, and a symbol and its **SymbolName** string are interchangeable. Explicit settings take priority over defaults, with the first applicable specification deciding a repeated option [16]. N2 violates the name-equivalence rule in the wrapper's classifier even though the engine itself handles the requested assumption.

A useful negative control prevents an overbroad recommendation. **Direction** does not appear in either of the two option lists above, even though it is supported by some other asymptotic-family functions. A call that happens to evaluate with an unadvertised option is not evidence that the option's semantics were honored. The current repository deliberately protects explicit direction,

branch, and budget requests on the package path [5]. This review therefore does not recommend blindly forwarding such contracts to a native engine.

4.4 Comparisons should test transformations, not only example functions

The newly reproduced failures suggest a more discriminating comparison method. Hold the mathematical function fixed and change only a language-level representation that ought to preserve the relevant request: move a setting from an explicit option to a configured default; replace a standard symbolic option name by its string spelling; rename the expansion variable to a legitimate option-named symbol.

Such tests isolate the interface from the mathematical engine. e^x is sufficient to expose N1, and $\sqrt{a^2}e^x$ under $a > 0$ is sufficient to expose N2. A catalogue containing hundreds of unrelated special functions can miss both because it never varies the request encoding.

The criterion is not unrestricted observational equivalence for arbitrary side-effectful Wolfram programs. The current compatibility plan explicitly allows preparation to have a different evaluation order from a direct native call. The proposed tests use well-defined literal forms and controlled callbacks, and state which counters or metadata are expected to agree. Section 8 develops those bounded obligations.

5 N1: configured defaults are ignored at the routing boundary

5.1 Minimal reproductions

Start from a freshly loaded pinned package. Save and restore the option list when performing this experiment in an existing notebook:

```
saved = Options[AsymptoticExpansion];
SetOptions[AsymptoticExpansion, "Backend" -> "Series"];
s = AsymptoticExpansion[Exp[x], {x, 0, 3}];
{s["Kind"], Normal[s]}
(* observed: {"Forward", 1 + x + x^2/2} *)

explicit = AsymptoticExpansion[Exp[x], {x, 0, 3},
  "Backend" -> "Series"];
{explicit["Kind"], Normal[explicit]}
(* observed: {"Native", 1 + x + x^2/2 + x^3/6} *)
Options[AsymptoticExpansion] = saved;
```

The configured selector and the explicit selector should select the same backend when no conflicting explicit selector is present. They do not. The public `Options` list advertises the selector and `SetOptions` accepts the change, but `dispatch` does not consult it.

A second experiment concerns the term goal:

```
saved = Options[AsymptoticExpansion];
SetOptions[AsymptoticExpansion, SeriesTermGoal -> 4];
s = AsymptoticExpansion[Exp[x], x -> 0];
{s["Kind"], Normal[s]}
(* observed: {"Native", 1} *)

explicit = AsymptoticExpansion[Exp[x], x -> 0,
  SeriesTermGoal -> 4];
{explicit["Kind"], Normal[explicit]}
```

```
(* observed: {"Forward", 1 + x + x^2/2 + x^3/6} *)
Options[AsymptoticExpansion] = saved;
```

The configured goal is not merely reported incorrectly. The router takes the leading-order native path and lets that engine use its own default, so the requested four-block package calculation is never attempted.

The alias exhibits the same design problem through another route:

```
saved = Options[AsymptoticExpand];
SetOptions[AsymptoticExpand, "Backend" -> "Series"];
s = AsymptoticExpand[Exp[x], {x, 0, 3}];
{s["Kind"], Normal[s]}
(* observed: {"Forward", 1 + x + x^2/2} *)
Options[AsymptoticExpand] = saved;
```

This is not a claim that every alias must have independent defaults. It is a mismatch between publishing an independently configurable `Options[AsymptoticExpand]` list and implementing a forwarding definition that never uses it.

5.2 Source-level cause

The dispatcher in `NativeCompatibility.wl` resolves an explicit selector but substitutes a literal `Automatic` when no selector appears:

```
backend = If[values === {}, Automatic,
  ReleaseHold[First[values]]];
```

This is independent of the current `Options[AsymptoticExpansion]`. The rule-form shape test separately checks whether `SeriesTermGoal` occurs syntactically in the request. It does not check a changed public default before deciding to use native leading-order semantics.

The alias is implemented as

```
Options[AsymptoticExpand] = Options[AsymptoticExpansion];
AsymptoticExpand[args___] := AsymptoticExpansion[args];
```

Copying the defaults at load time does not make later alias settings part of the forwarded call. These three observations are one finding because they share a root cause: the routing layer does not have a well-defined effective option environment.

5.3 Why the mathematical answer can be valid and the API still wrong

For $x \rightarrow 0$, the expression 1 is a valid leading approximation to e^x . Thus the term-goal example does not contradict an asymptotic theorem. However, an API request is more than the proposition that its output approximates the source at some unspecified accuracy. The configured request asks for a particular backend or a particular amount of expansion work.

Changing the backend also changes the object kind and cutoff interpretation. A caller may subsequently use analytic operations or inspect native data based on the configuration it set. Silent routing away from that configuration undermines reproducibility even when every returned coefficient happens to be correct.

The severity is therefore a medium interface-correctness defect with potentially broad effects on

library wrappers and notebook-level configuration. It should not be inflated into a high-severity false-remainder report without an actual false remainder witness.

5.4 A safe default-resolution policy

The tempting repair is to append *all* public defaults to every request. That is unsafe. The public option list is a union containing native-only keys, and its default values come from different engines. Appending them all would create explicit native-option routing on ordinary analytic calls and could activate protected package constraints that the caller never supplied.

A bounded compatibility policy is to preserve the current baseline behavior while promoting *changed* public defaults into the effective request. Capture the constructor's and alias's baseline option rules when the native layer loads. At an entry point, append only rules that differ from the relevant captured baseline. Keep explicit options first. For the alias, a defensible precedence is

explicit settings > alias overrides > constructor overrides > baseline routing policy.

This precedence is a proposed documented policy, not a hidden assumption about every possible alias convention.

The patch candidate uses that approach so ordinary existing calls keep their current behavior. It preserves delayed rules without executing their right-hand sides merely to compare or append defaults. It also preserves the original held user arguments separately from the effective request. A longer-term implementation could expose the default provenance directly instead of relying on a load-time baseline comparison.

There is an unavoidable information limit: `SetOptions` does not generally remember whether a value equal to the original default was explicitly reasserted. A baseline-difference policy cannot recover that lost history. It should therefore promise to honor changed defaults, not to reconstruct every prior configuration action. Callers requiring an explicit baseline-valued routing constraint must still supply that option explicitly, unless the package introduces a separate configuration API.

5.5 Acceptance criteria

The repair should make configured and explicit backend selectors agree for both public spellings, restore ordinary behavior after resetting defaults, preserve explicit-option precedence, and honor configured term goals without accidentally injecting all native options. It must also cover delayed defaults with counters, nested explicit option containers, and a configured package-only budget that remains binding.

Run these tests in a fresh kernel and restore `Options` after every scoped test. Otherwise, a failing default test can contaminate later tests and falsely implicate unrelated mathematical functionality.

6 N2: equivalent option spellings receive incompatible treatment

6.1 The observed native differential

The native control computes a real simplification under a string-spelled assumption option:

```
Normal[Series[Sqrt[a^2] Exp[x], {x, 0, 2},
  "Assumptions" -> a > 0]]
(* observed: a + a x + (a x^2)/2 *)
```

The corresponding wrapper call does not reach an equivalent computation:

```
AsymptoticExpansion[Sqrt[a^2] Exp[x], {x, 0, 2},
  "Assumptions" -> a > 0]
(* observed: Failure["NativeOptionConflict", ...] *)
```

This is a particularly useful witness because the native assumption has an observable effect: it replaces $\sqrt{a^2}$ by a . The result is not merely evidence that the engine ignored an unfamiliar option and continued.

Official `OptionValue` documentation treats a symbol and its `SymbolName` string interchangeably, and ignores contexts when comparing option names [16]. Standard Wolfram callers therefore have a reasonable expectation that the wrapper accepts the alternate spelling when the underlying backend does.

6.2 Exact held syntax is being used for semantic identity

The native layer extracts keys as held expressions. For a symbolic assumption key it records `HoldComplete[Assumptions]`; for the string spelling it records `HoldComplete["Assumptions"]`. These two expressions are structurally different.

The package key set and the native engine key sets use the symbolic forms. The string key is first classified as outside the package key set. It is then not found in the symbolic native option lists, so backend selection returns `NativeOptionConflict`. The error message suggests that no engine supports all supplied options, even though the control demonstrates otherwise.

Holding option syntax is correct for evaluation safety. The mistake is to use the same held syntax as the sole semantic identity of an option. The repair must preserve the former while correcting the latter.

6.3 Canonicalize identity without evaluating values

Build a finite dictionary from known option names to their canonical held keys. For a symbolic key, compute its unqualified `SymbolName`; for a string key, use the string directly. Look up that name in the known dictionary. An unknown key should remain unknown rather than being silently interned or accepted.

Only the *classification key* needs this normalization. The original option rule, including a delayed value and its original spelling, can remain in the held request and be forwarded to the engine. This avoids three unnecessary risks: evaluating an option value early, executing arbitrary text with `ToExpression`, and losing original-request provenance.

The candidate patch applies this dictionary at `nativeOptionKeys`. It does not claim to implement every possible custom-key spelling or every malformed native input. In particular, the wrapper's explicitly string-named backend selector needs its own clearly documented syntax and positional treatment before a blanket equivalence rule is added to selector stripping.

The dictionary should be derived from the supported option sets at a known initialization point. If the package permits option-set changes, specify whether the dictionary is recomputed. Changing an existing option's value and adding an entirely new supported option are different operations; the former does not require inventing a new classifier key.

6.4 Tests beyond the smallest example

A focused regression matrix should compare symbolic and string spellings for `Assumptions`, `Analytic`, and `SeriesTermGoal`; compare recognized context-qualified symbolic spellings where

`OptionValue` treats them identically; and exercise nested option lists, repeated keys, and `RuleDelayed` values.

The test should check more than successful evaluation. Compare native result, normalized expression, selected backend, kind, applicable routing reason, and callback counts. For package-only mode, ensure that a standard string-spelled common option is not falsely labeled a native-only unsupported option. For genuinely unsupported options, keep a failure rather than converting this repair into permissive option dropping.

7 N3: specification parsing confuses variable names with option names

7.1 An expansion variable named Method is legal

Wolfram symbols can be used as algebraic variables even when the same symbol is also used as an option name elsewhere. The native control is straightforward:

```
Asymptotic[Exp[Method], Method -> 0]
(* observed: 1 *)
```

The explicit wrapper delegates successfully:

```
s = AsymptoticExpansion[Exp[Method], Method -> 0,
  "Backend" -> "Asymptotic"];
s["NativeResult"]
(* observed: 1 *)
```

But its metadata and application interface do not describe that successful request:

```
s["ExpansionSpecifications"]
(* observed: {} *)
s["Variable"]
(* observed: Missing["MultipleOrUnresolvedVariables"] *)
s[2]
(* observed: Failure["NativeVariables", ...] *)
```

This is not an ambiguous multivariate problem. There is one explicit native expansion specification. The native result is retained, so the observed defect is metadata and wrapper usability rather than corruption of the native coefficient computation.

The constant leading term makes the example unusually easy to inspect: numerical application would return one for any substituted value if the variable were correctly recognized. No special-function singularity or numeric solver is involved.

7.2 The classifier ignores positional grammar

The inspected definition recognizes a rule as a specification only when its symbolic left-hand side is *not* a known public option key:

```
nativeSpecificationQ[
  HoldComplete[(Rule | RuleDelayed)[key_Symbol, _]]] :=
  ! MemberQ[First /@ Options[AsymptoticExpansion],
    Unevaluated[key]];
```


This predicate is reused for collecting specifications, extracting options, selecting native backends, and constructing result metadata. Since `Method` is a native option key in the public union, the positional specification `Method -> 0` is classified as an option.

The direct native call does not need this global name test to determine the role of the argument. Its grammar supplies that role. An option called `Method` after an expansion specification is different from an expansion specification whose variable is `Method`.

The bug also demonstrates why string canonicalization alone is insufficient. N2 asks the classifier to identify semantically equivalent *option keys*; N3 requires the parser to decide whether an argument is an option at all. Those phases must occur in that order: parse role first, normalize an option key second.

7.3 A positional prefix parser

For the currently supported ordinary native forms, split the request tail into a positional specification prefix and an option suffix. The first admissible expansion specification is positional even when its variable's name collides with an option name. Successive native list specifications remain part of the prefix. Nested lists of rules after that prefix remain option containers.

The supplied candidate defines a held split and uses it consistently in option collection, native shape selection, backend selection, package preparation, and native result metadata. It deliberately leaves the source and option values unreleased. The original helper can remain for historical internal uses, but it must not independently make contradictory role decisions in a path that uses the new split.

This is a bounded grammar repair. A full grammar still needs explicit decisions for every native form admitted by the supported kernel: multivariate list specifications, computed argument containers, sequences, rule-delayed specifications, malformed inputs, and native calls with nonliteral variable structures. The candidate includes a conservative prefix rule rather than claiming that a short predicate exhausts that grammar.

7.4 Why role information should be retained

It is inefficient and error-prone to rediscover argument roles several times from raw syntax. The current implementation selects specifications in more than one helper, and the same name-based mistake propagates to all of them. Parsing once would let the request carry the source, ordered held specifications, option containers, and wrapper selector as separate fields.

For this finding, the necessary invariant is small: after successful delegation of a one-variable literal native request, the stored recognized specification and variable must correspond to the actual delegated specification, unless a documented representation limitation applies. A variable's spelling is not such a limitation.

Metadata are not incidental decoration. They govern numerical application and provide the audit trail used to understand what was computed. Lossless "NativeResult" storage alone is therefore not complete wrapper fidelity.

8 A request-semantics framework for the three repairs

8.1 The missing effective-request layer

The three defects can be described without any special-function mathematics. Let H be a held user request, D the relevant default environment, and C the captured evaluation context. Let

$$E(H, D, C)$$

be an effective request containing its source, positional specifications, semantic option identities, still-held values, and routing constraints. Let R select the package or native route, and let B perform the selected backend computation.

A wrapper can preserve the backend result perfectly and still be incorrect if it computes

$$B(R(E(H, D, C)))$$

from the wrong E . N1 omits relevant parts of D . N2 uses exact syntax where semantic option identity is required. N3 confuses an argument's role with its symbol name. Adding more coefficient formulas does not repair any of these errors.

Design principle 1 (Role before identity; identity before value). *First determine whether an argument is a specification, an option container, or a wrapper selector. Then normalize an option's identity. Resolve its value only when the selected route requires that value, under the appropriate evaluation context.*

This principle is a specific refinement of the repository's existing request-normalization proposals, justified by the current failures. It is not presented as a newly discovered need for an entirely generic intermediate representation.

8.2 Three bounded invariants

Default equivalence. For an option whose changed configured default is meant to govern a request, an omitted explicit setting should agree with supplying that effective default explicitly, unless a documented routing policy intentionally distinguishes them. Result kind and order interpretation belong in that comparison, not just the finite coefficients.

Option-spelling equivalence. Once an argument has been identified as an option, documented equivalent names must receive the same compatibility classification. Their raw spellings may remain distinct in `OriginalArguments`; they should not select different backends or falsely change support status.

Variable-renaming equivariance. Renaming a free expansion variable in a literal, well-scoped request should rename its specification and variable metadata, without turning the specification into an option. This is a syntactic hygiene property, not a demand to ignore actual definitions attached to arbitrary symbols.

For a pure literal request, these can be checked with small commuting diagrams: transform the input then wrap and compute, or wrap and compute then transform the appropriate output fields. The comparison must respect the selected order convention. Comparing only `Normal` can hide a kind or metadata defect, as N3 illustrates.

8.3 Do not overpromise effect equivalence

Wolfram Language inputs can contain delayed rules, computed option lists, assigned symbols, and arbitrary side effects. The existing compatibility plan already distinguishes literal native delegation from package preparation and records that their evaluation order need not be identical [5]. The repair must preserve that evidence boundary.

The desired guarantee is not “every arbitrary Wolfram program executes with identical traces.” It is narrower: classification does not execute option right-hand sides; a consumed common delayed value is not consumed again merely for metadata; source programs are not replayed just because a representation fallback occurs; and ordinary literal native requests preserve their intended arguments and conditions.

A counter on a deliberately delayed option is a useful regression. It is not a reason to assert a universal theorem about side-effect ordering. Conversely, a test whose native engine itself resolves a delayed option more than once should use that native behavior as its explicit baseline rather than impose an invented one-call rule on every backend.

8.4 A practical acceptance matrix

The following matrix adds rows that the existing mathematical examples do not systematically exercise:

Dimension	Representative cases	Required observation
Entry point	<code>AsymptoticExpansion</code> ; held alias <code>AsymptoticExpand</code>	Same configured policy or an explicitly documented alias policy.
Default source	Baseline; changed constructor default; changed alias default; explicit override	Chosen backend, kind, order, goal, and reset behavior.
Option spelling	Symbol; standard string; recognized alternate context; unknown string	Equivalent keys classify equally; unknown keys still fail.
Option container	Flat rules; nested lists; computed lists; <code>Sequence</code> ; delayed values	No premature value execution or duplicate replay introduced by classification.
Variable spelling	Fresh symbol; <code>Method</code> ; <code>Assumptions</code> ; another recognized option symbol	Positional role and one-variable metadata remain correct.
Specification form	Rule; list; successive lists; symbolic/complex center; malformed form	The actual delegated syntax and ordered specifications remain inspectable.
Backend outcome	Native series; ordinary expression; conditional/list output; unresolved native call	No invented analytic remainder; truthful computed/unresolved status.
Protected request	Explicit branch, direction, or package budget; callable/inverse source	No silent loss of those constraints in pursuit of a native answer.

This matrix is a focused extension to B04, not a claim to replace the full native coverage plan. It should be implemented with independent direct-native controls whenever the intended native semantics are known.

9 Performance, representation, and development opportunities

9.1 P-N1: native normalization is eagerly materialized

The inspected native constructor always performs

```
result = Block[{$Assumptions = ambient}, ReleaseHold[call]];
normal = Normal[result];
```

and stores both "NativeResult" -> **result** and "Expression" -> **normal**. Thus a caller requesting only the preserved native object has already paid for normalization. This work is separate from the old optional sparse-to-dense **SeriesData** exporter, which has its own existing repair history [6, 4].

For the simple family

$$f(x) = \frac{1}{1-x}, \quad S_n(x) = \sum_{j=0}^n x^j + O(x^{n+1}),$$

the native **SeriesData** stores a coefficient sequence, while **Normal** constructs an explicit sum of powers. Both representations grow with n ; this source argument does not establish an asymptotic complexity-class separation or an exponential blow-up. It does establish that an additional expression is constructed even when the user does not request it.

The successful experiment used the pinned unmodified standalone, explicit native **Series** mode, and the same function and order in the direct native control. Both **NativeResult** equality and normalized-expression equality were **True** in each row:

Native order	Native ByteCount	Normal ByteCount	Wrapper ByteCount
100	2,632	8,008	15,416
1,000	24,360	80,136	109,056

At order 1,000, the wrapper's measured expression size is about 4.48 times the native value's. The extra normalized view is the dominant additional expression visible in this accounting, but the wrapper also contains metadata. These values are not a measurement of physical heap duplication, shared internal storage, or peak resident memory. No timing comparison was successfully recorded.

The archive includes **benchmark_native.wl**, which can reproduce matched size observations and additionally record labeled wall-clock observations. Those optional timing fields are not part of the successful evidence above. A single warm timing must not be described as a stable speedup, and changing to a lazy representation still requires the evaluation-timing review discussed below.

A targeted improvement is a lazy native normalization view. Preserve the complete **NativeResult** immediately; compute **Normal[NativeResult]** when **Normal[s]**, **s["Expression"]**, or numerical application actually requests it. Decide explicitly whether that value is memoized. Preserve display behavior, conditional/infinite outputs, and compatibility of restored objects. A direct private helper that lazily normalizes a native object is preferable to distributing special cases through every analytic operation.

There is an evaluation caveat: delaying **Normal** can change when surviving symbols evaluate. The current documentation already notes that native metadata do not freeze all external definitions. A lazy design should document this timing and should not be marketed as transparent for every side-effectful expression. An eager opt-in snapshot or a held normalization view may be appropriate for users who need a specific evaluation moment.

9.2 Budget only the work actually owned by the wrapper

The public package budget **"MaxTerms"** is intentionally not silently passed into an unsupported native budget. This is correct. The new wrapper nevertheless owns some work that is not native coefficient computation: preparing argument containers, normalizing keys, storing provenance, and constructing normalization views.

The near-term proposal is not another undifferentiated global budget; that broader issue is already P06. Instead, measure the native wrapper's own preparation and materialization cost separately. Report native engine time, wrapper preparation time, and optional view cost when profiling a

problematic case. A native unresolved result is not a successful fast computation, and a wrapper failure must remain in the benchmark table rather than being omitted.

For pure option classification, a finite name dictionary avoids repeated string-to-symbol work. For specification parsing, a single request split avoids repeated traversal and inconsistent role decisions. These are primarily correctness simplifications. Their speed benefit should be measured rather than presumed to dominate symbolic computation.

9.3 Native operations without analytic promotion

The repository already refuses to treat a native object as an analytic jet. That is sound but can surprise users who see a `GeneralizedSeries` head and expect the same algebra everywhere. A useful, narrowly scoped development direction is explicit native operations that operate on `NativeResult` and preserve a native result contract, rather than routing them through the analytic error algebra.

For example, native composition of suitable `SeriesData` objects could delegate to `ComposeSeries`, and native reversion could delegate to `InverseSeries`. This is not a proposal to infer a real-domain or logarithmic error theorem. The output would remain native, preserve the delegated operation, and fail normally when the native object lacks the appropriate structure. An operation-specific native adapter would be more predictable than silently stripping `Normal` and rebuilding an exact analytic object.

This direction overlaps the already recorded native-operations roadmap, so its incremental recommendation is concrete: start with structure-preserving native operations whose input and output contracts can be directly compared to their built-ins. Do not first create a universal conversion from native expressions to analytic envelopes.

9.4 The path toward complete native coverage

The accepted full-superset objective remains open, and the current plan already identifies the missing second-backend search. Merely trying both native heads is not a complete solution. It raises questions about shared versus conflicting order conventions, unsupported options, repeated side effects, result quality, and when a remaining native call counts as unresolved.

The immediate route to more defensible coverage is incremental. First make the already intended native requests survive harmless option and variable encodings. Next distinguish a backend's unsupported or unresolved result from a result whose shape is simply unfamiliar. Finally define a bounded second-backend policy on an effective, replay-safe request, preserving the first attempt and its reason for rejection.

A structural preservation theorem would be conditional: if a selected native engine succeeds on the effective request and the returned expression is representable by the native wrapper, wrapping preserves that expression and its native contract. Universal *selection* coverage is a separate property. N2 fails before the first premise is even reached; N3 shows that expression preservation alone is weaker than metadata fidelity.

10 Patch candidate and artifact design

10.1 What the candidate changes

The archive contains a source patch specification and a fail-closed Python patcher. The target is only `src/Kernel/NativeCompatibility.wl`. The patcher checks the repository revision by default and verifies exact occurrence counts for every source anchor before making any change. Its

default action prints a unified diff; `-apply` writes the canonical module atomically. It does not push commits, modify a remote repository, or generate checksum files.

The candidate makes three focused changes. It supplies changed constructor and alias defaults to dispatch with explicit settings first. It canonicalizes known option identities for classification. It replaces the repeated name-based specification selections by one consistent positional-prefix split. The helper source is included separately for inspection, and the JSON specification is the patcher's authoritative input.

After applying the canonical source change, regenerate the standalone distribution using the repository's own build script. Editing only the root generated file would create a maintenance divergence that the next build could erase.

10.2 What it deliberately does not change

The candidate does not alter real coefficient admission, logarithmic tail inference, exponent comparison, inverse coefficient construction, branch proofs, numerical solvers, or interval arithmetic. It does not make `Normal` lazy. It does not add a second-backend search, expand protected native fallback, or establish complete native grammar coverage.

The default policy also remains a policy choice. A stricter alternative would make the alias share only constructor defaults and reject or discourage separate alias configuration. That would require changing its advertised option behavior and documenting the incompatibility. The supplied candidate instead makes the existing separate alias option list useful under an explicit precedence rule.

10.3 Validation status of the candidate

The candidate was successfully executed in five focused probes on the same Wolfram kernel. The eight replacement anchors had the expected occurrence counts $\{1, 1, 1, 1, 1, 2, 1, 1\}$, and the canonical native module appeared once inside the pinned standalone. The candidate definitions were inserted before loading that standalone. The configured backend and alias backend then returned native order-three expansions; the configured four-block goal returned the intended four-block analytic expansion; the string assumption returned the same analytic cutoff-two expression as its symbolic spelling; and the variable `Method` retained its specification, variable metadata, and numerical application.

The successful probe used the same replacements and helper definitions as the supplied candidate, with compact formatting and without the explanatory comments. It was not a test of a separately rebuilt checkout or of the Python command-line installer. A subsequent split run successfully passed all 68 tests in `NativeAutomatic.wlt` and `NativeCompatibility.wlt`, with the same optimizer message-text caveat as the baseline. Calls for all four files together or the remaining two files encountered connector failures without a completed report. The full twenty-five-case candidate regression file also remains unexecuted in this session. No wider patched-suite pass count is claimed.

The Python patcher's fail-closed transformation logic is independently testable without a Wolfram kernel. Passing those Python checks verifies source transformation safeguards, not Wolfram evaluation semantics. The candidate still needs acceptance on the exact locally regenerated standalone, completion of the remaining twenty-two existing native tests in this environment, and the full new default, spelling, and variable-role matrix.

This boundary is intentional. A repair with five successful probes, 68 passing existing tests, and transparent remaining acceptance work is safer than calling it production-ready from this subset alone. The baseline defects do not depend on the candidate: their observed outputs and source causes stand independently.

10.4 How to reproduce and integrate

With an inspected local checkout at the audited revision, run:

```
python code/patch_native_boundary.py /path/to/Asymptotic
python code/patch_native_boundary.py /path/to/Asymptotic --apply
cd /path/to/Asymptotic
python validation/build_standalone.py
```

Then load the unmodified or regenerated standalone in a fresh Wolfram kernel and run the provided characterization script and regression file. The archive includes an existing-test runner for the four native files used in this review. Baseline characterizations report what currently happens; acceptance regressions express the desired repaired behavior and are expected to fail on affected baseline cases.

Do not run baseline and patched comparisons in a notebook with unrelated persistent `SetOptions` changes. Save and restore option lists inside each test. Record the exact kernel, source revision or local modification status, the selected test files, counts, and unexpected messages. A patch that fixes N1 but forces all ordinary requests onto the native path has failed even if its new isolated backend test passes.

11 Validation results, limits, and development sequence

11.1 Existing native-interface tests executed here

The following four repository files were run against the pinned standalone in Wolfram Language 15.0.0 for Linux:

```
src/Tests/NativeAutomatic.wlt
src/Tests/NativeCompatibility.wlt
src/Tests/NativeContracts.wlt
src/Tests/NativePresentation.wlt
```

The reported result was **90 successes and 0 failures**. The evaluator also emitted repeated `MinValue::lpsub` and `MaxValue::lpsub` messages whose message text was unavailable, followed by a general message notice. The returned report still recorded zero failed tests. The evidence file retains that caveat; this was not a completely silent run.

This subset exercises explicit/native routing, selected automatic fallback, held argument behavior, analytic-contract refusal, and presentation. It does not cover all mathematical engines or prove the new input-superset objective. Passing it is compatible with N1–N3 because the reproduced dimensions are outside the tested cases.

The N1–N3 observations were separate, focused package evaluations. Their outputs are recorded as normalized transcriptions of the successful tool results, with exact source and kernel identity. They are not represented as a single machine-exported native test report. This distinction makes the evidence straightforward to audit and replay.

11.2 Useful negative results

Several tempting claims were not promoted to findings. An explicit `SeriesTermGoal -> Automatic` in a rule-form built-in `Asymptotic` request itself produces an order-specification diagnostic; it is not a valid native-success counterexample against the wrapper. Explicit direction protection is documented and must not be removed simply because an unadvertised native option happens to

be tolerated. One delayed-option probe had the same observed callback count in the native and wrapper paths, so it does not establish duplicate evaluation.

No new unsound interval certificate was reproduced. The inspected certificate source includes the modern domain and sharp-enclosure safeguards. The existing register’s remaining analytic obligations are not declared closed merely because this review focused on another part of the system.

These negative results matter because they keep a code review from becoming a catalogue of plausible-sounding but unsupported faults.

11.3 A focused sequence with objective exit criteria

First, lock down request equivalence. Add the smallest N1–N3 regressions before changing dispatch. Preserve current successful analytic and native controls, including the distinction between exclusive package cutoff and native order. Exit only when the altered input encodings produce the intended equivalent requests and metadata.

Second, integrate a single held role parser and default policy. Review its treatment of nested options, sequences, delayed values, conflicting selectors, and option-named variables. Keep original syntax separate from effective settings. Run the existing four native files and the new regressions on both the canonical and regenerated standalone entry points.

Third, measure optional native materialization. Use the supplied benchmark with matched outputs, multiple orders, cold and warm observations, and an explicit definition of memory metrics. Introduce lazy normalization only after evaluating its symbol-evaluation timing and saved-object compatibility.

Fourth, expand native coverage by semantic dimension. Add documented native option forms and source structures, not only new function names. A second-backend search should be implemented against a defined effective request and should retain failed-attempt provenance. Keep protected branch and budget contracts binding.

These steps complement, rather than replace, the existing prioritized mathematical repair register. They are the additional work justified by this review’s current native-boundary evidence.

12 Conclusion

AsymptoticAnalysis has a substantial specialized calculus and a sensible separation between package analytic objects and native results. Its current value is not accurately described as duplicating **Series**, nor is its native coverage accurately described as complete. The most useful next improvement is to make request interpretation reliable before adding more delegated cases.

Three small native experiments expose a common architectural weakness: dispatch currently confuses raw syntax with effective semantics. Configured defaults, equivalent option names, and positional variable roles need to be interpreted consistently. Repairing that boundary can improve correctness and usability without touching the mature coefficient and certificate engines.

The evidence is deliberately bounded: ninety existing tests passed on the baseline, three new interface defects were reproduced, and a concrete candidate corrected five focused probes and passed 68 existing tests. Two matched expression-size measurements support the native materialization discussion. No full-suite, complete candidate-matrix, runtime-speedup, or peak-memory claim is made. The archive contains enough code and provenance to turn that candidate into a reproducible integration decision rather than another unaudited recommendation.

A Archive contents and evidence interpretation

File or directory	Purpose
article/asymptotic_native_boundary_audit.tex	Self-contained article source with pinned source links and official references.
article/asymptotic_native_boundary_audit.pdf	Compiled article.
code/load_pinned.wl	Load the trusted pinned standalone, or an explicitly supplied local file.
code/characterize_boundary.wl	Replay and report the N1–N3 baseline characterizations.
code/NativeBoundaryRegressions.wlt	Twenty-five desired-behavior regressions and controls; this full file was not executed here.
code/run_existing_native.wl	Run the same four existing native-interface test files.
code/native_boundary_patch.json	Authoritative exact source replacements and helper definitions.
code/native_boundary_helpers.wl	Readable helper excerpt; do not load it alone as a complete patch.
code/patch_native_boundary.py	Revision- and anchor-checked dry-run/apply transformation.
code/apply_patch_in_memory.wl	Applies the same patch specification to an inspected standalone source string before loading.
code/test_patch_transform.py	Independent tests of the patcher’s source-edit safeguards.
code/benchmark_native.wl	Reproduces the size experiment and can add timing observations; only size results were recorded here.
evidence/observations.json	Normalized successful baseline and five-probe candidate observations, with execution limits.
evidence/novelty_crosswalk.csv	New findings versus closest prior work and current scope.
evidence/source_coverage.csv	Inspection and execution levels, without pretending that all modules were re-proved.
evidence/prior_review_scan.csv	Paths and counts for the targeted prior-article search.
evidence/python_patch_tests.txt	Successful six-test Python transformation-safeguard report, not a Wolfram acceptance report.
README.md, build.sh	Reproduction instructions, limitations, and article build command.

The archive intentionally contains no checksum files, no font files, no upstream repository clone, and no LaTeX auxiliary build products. The commit identifier pins the review; it is not a substitute for checking a downloaded executable source before loading it.

B Reference notes

Repository references below all refer to the audited commit, not a moving `main` branch. Official documentation was consulted during this review; runtime comparisons separately identify the

observed kernel. Source descriptions and proposed designs are distinguished throughout so that an implementation plan is not mistaken for completed native evidence.

References

- [1] Vladimir Reshetnikov, *AsymptoticAnalysis README*, audited revision. Pinned README.
- [2] Vladimir Reshetnikov, *Record successful loading from the published AsymptoticAnalysis URL*, commit 6687962f3c85. <https://github.com/VladimirReshetnikov/Asymptotic/commit/6687962f3c858a4f93623cfc496f33e35c6763d4>.
- [3] *External code-review collection and wave indexes*, audited revision. Review collection.
- [4] *Code review implementation status*, audited revision. Current consolidated register.
- [5] *Native expansion compatibility: required coverage and implementation plan*, audited revision. Native compatibility plan.
- [6] *NativeCompatibility.wl*, audited revision. Canonical dispatcher source. Particularly the option setup and alias, held entry, specification classifier, automatic shape test, and final native constructor.
- [7] *AsymptoticAnalysis.wl*, canonical modular entry point, audited revision. Core source and public contracts.
- [8] *InverseCertificates.wl*, audited revision. Exact interval-certificate implementation.
- [9] *NativeContracts.wlt*, audited revision. Native contract refusal and control tests.
- [10] *NativeAutomatic.wlt*, audited revision. Focused automatic-routing tests.
- [11] Wolfram Research, *Series*. <https://reference.wolfram.com/language/ref/Series.html>.
- [12] Wolfram Research, *SeriesData*. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [13] Wolfram Research, *InverseSeries*. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [14] Wolfram Research, *Asymptotic*. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [15] Wolfram Research, *AsymptoticSolve*. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [16] Wolfram Research, *OptionValue*. Especially the documented context-insensitive comparison of option names, symbol/string equivalence, explicit/default precedence, and held-value form. <https://reference.wolfram.com/language/ref/OptionValue.html>.
- [17] Wolfram Research, *Options*. <https://reference.wolfram.com/language/ref/Options.html>.